LoRA MLP + Attention (Low Rank)

This notebook mirrors the full-run LoRA pipeline and focuses on low-rank target-module ablations for attention-only vs attention+MLP adapters.

```
from pathlib import Path
import sys
import pandas as pd
import torch
from IPython.display import display
```

Colab setup and project root

```
try:
    from google.colab import drive
    IN_COLAB = True
except ImportError:
    IN_COLAB = False

if IN_COLAB:
    drive.mount('/content/drive')
    CANDIDATE_ROOT = Path('/content/drive/MyDrive/DL_Final_Project')
    PROJECT_ROOT = CANDIDATE_ROOT if CANDIDATE_ROOT.exists() else Path.
else:
    cwd = Path.cwd().resolve()
    if cwd.name == 'notebooks':
        PROJECT_ROOT = cwd.parent
    elif cwd.name == 'LoRA_study' and cwd.parent.name == 'notebooks':
        PROJECT_ROOT = cwd.parent.parent
    else:
        PROJECT_ROOT = cwd

print('IN_COLAB:', IN_COLAB)
print('PROJECT_ROOT:', PROJECT_ROOT)
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))
```

```
Drive already mounted at /content/drive; to attempt to forcibly remoun
IN_COLAB: True
PROJECT_ROOT: /content/drive/MyDrive/DL_Final_Project
```

```
import importlib.metadata as md
import importlib.util
import subprocess


def _parse_version_tuple(version_str: str) -> tuple[int, ...]:
    parts = []
    for token in version_str.split('.'):
        digits = ''.join(ch for ch in token if ch.isdigit())
        parts.append(int(digits) if digits else 0)
    return tuple(parts)


required_torchao = (0, 16, 0)
if importlib.util.find_spec('torchao') is not None:
    v_str = md.version('torchao')
    print(f'Found torchao=={v_str}')
    if _parse_version_tuple(v_str) < required_torchao:
        print('Uninstalling incompatible torchao...')
        subprocess.check_call([sys.executable, '-m', 'pip', 'uninstall'
        raise SystemExit('Removed incompatible torchao. Restart runtime
    print('torchao is compatible.')
else:
    print('torchao is not installed (this is fine).')
```

```
torchao is not installed (this is fine).
```

```
from src.gen_lora_study_utils import (
    set_seed,
    load_split,
    apply_sanity_subset,
    cap_validation_rows,
)
from src.lora_best_candidates_full_run_utils import build_configs_fro
from src.lora_tuning_block_utils import count_trainable_params_for_co
```

Runtime controls (same style as full-run notebook)

```
save = True

# For the sweep, keep this False so you do not run test inference for
```

```
# After choosing the best validation config, rerun only that config w
generate_submission = True #True

# Empty means "all ablations" (same behavior as submission_ablation_i
submission_ablation_ids = []
save_val_predictions = True
val_prediction_ablation_ids = []#['score_lora_sft', 'score_lora_optio

sanity_check = True
n = 512  # ignored when sanity_check=False

seed = 42
max_val_examples = 256 #None
top_k = 5
batch_size = 4
clear_cuda_between_runs = True

# For option_scoring-only runs, loss is a smoother early-stopping sig
# For mixed option_scoring + SFT runs, use 'accuracy' instead; see no
early_stopping = None
epochs_per_val_check = None
early_stopping_metric = 'loss'
early_stopping_val_examples = 128 #None

DATA_DIR = PROJECT_ROOT / 'data'
IMAGE_DIR = DATA_DIR / 'images' / 'images'
OUT_DIR = PROJECT_ROOT / 'outputs' / '03_dora_scoring'
ADAPTER_DIR = OUT_DIR / 'adapters'
MODEL_ID = 'HuggingFaceTB/SmolVLM-500M-Instruct'
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'

torch_dtype = (
    'bfloat16'
    if torch.cuda.is_available() and torch.cuda.is_bf16_supported()
    else ('float16' if torch.cuda.is_available() else 'auto')
)

print('CUDA available:', torch.cuda.is_available())
if torch.cuda.is_available():
    print('GPU:', torch.cuda.get_device_name(0))
print({
    'save': save,
    'generate_submission': generate_submission,
    'submission_ablation_ids': submission_ablation_ids,
    'save_val_predictions': save_val_predictions,
```

```
    'val_prediction_ablation_ids': val_prediction_ablation_ids,
    'sanity_check': sanity_check,
    'n': n,
    'device': DEVICE,
})
set_seed(seed)
```

```
CUDA available: True
GPU: NVIDIA A100-SXM4-80GB
{'save': True, 'generate_submission': True, 'submission_ablation_ids':
```

```
train_df = load_split(DATA_DIR, 'train')
val_df = load_split(DATA_DIR, 'val')
test_df = load_split(DATA_DIR, 'test')

train_df = apply_sanity_subset(train_df, sanity_check, n, seed)
val_df = apply_sanity_subset(val_df, sanity_check, n, seed)
test_df = apply_sanity_subset(test_df, sanity_check, 1008, seed)
val_df = cap_validation_rows(val_df, max_val_examples)

print('train rows used:', len(train_df))
print('validation rows used:', len(val_df))
print('test rows used:', len(test_df))
print('IMAGE_DIR exists:', IMAGE_DIR.exists())
```

```
train rows used: 512
validation rows used: 256
test rows used: 1008
IMAGE_DIR exists: True
```

Candidate ablation spaces

The key differences are low-rank settings and target modules. Input/data ablations are toggled directly in each candidate config.

```
# Single best-attempt config after the SFT masking / generation / opt

input_ablation_1 = {
    # The strategy note says the original images are small and recomm
    # Use 512 for this run. If you later find many images are non-squ
    # and control processor resolution instead.
    'image_size': 200,
```

```
    # Keep augmentation light. These images likely contain diagrams/te
    'enable_image_augmentation': True,
    'brightness_jitter': 0.03,
    'slight_rotation_deg': 0.5,

    # Use metadata because subject/grade/topic may help route the reas
    'include_metadata_in_prompt': True,
    'metadata_fields': ['subject', 'grade', 'topic'],
    'metadata_header': 'Metadata',

    # With output_format='letter_only', this is mostly harmless/unuse
    # but keep it simple in case other code references it.
    'answer_prefix_text': 'Answer:',
    'question_label': 'Question:',
    'choices_label': 'Choices:',
    'instruction_prefix': '',
}
input_ablation_2 = {
    # The strategy note says the original images are small and recomme
    # Use 512 for this run. If you later find many images are non-squa
    # and control processor resolution instead.
    'image_size': 384,

    # Keep augmentation light. These images likely contain diagrams/te
    'enable_image_augmentation': True,
    'brightness_jitter': 0.03,
    'slight_rotation_deg': 0.5,

    # Use metadata because subject/grade/topic may help route the reas
    'include_metadata_in_prompt': True,
    'metadata_fields': ['subject', 'grade', 'topic'],
    'metadata_header': 'Metadata',

    # With output_format='letter_only', this is mostly harmless/unuse
    # but keep it simple in case other code references it.
    'answer_prefix_text': 'Answer:',
    'question_label': 'Question:',
    'choices_label': 'Choices:',
    'instruction_prefix': '',
}
prompt_structure = 'question_first'
context_mode = 'hint_only'
choice_format = 'letter_dot'
```

```
# Important: use letter-only with option scoring.
# This makes each candidate just "A", "B", "C", etc.
output_format = 'letter_only'

# Option scoring does not rely on free generation, but keep these sane
max_new_tokens = 4
decoding_strategy = 'greedy'
num_beams = 1
parse_rule = 'strict_first_letter'

# Option scoring is slower than SFT because it scores every candidate
# Start with 12 rather than 25; early stopping can still stop sooner.
epochs = 3

ablation_space = {

    'dora': {
        # Main change: use multiple-choice candidate scoring rather th
        'training_objective': 'option_scoring',

        'prompt_structure': prompt_structure,
        'context_mode': context_mode,
        'choice_format': choice_format,
        'output_format': output_format,
        'max_new_tokens': max_new_tokens,
        'decoding_strategy': decoding_strategy,
        'num_beams': num_beams,
        'parse_rule': parse_rule,

        # DoRA + attention/MLP targets.
        # r=4 should stay safely below your 5M trainable-parameter cap
        # attention behavior and internal MLP representations.
        'lora_r': 4,
        'lora_alpha': 8,
        'lora_dropout': 0.03,
        'target_modules': [
            'q_proj', 'k_proj', 'v_proj', 'o_proj',
        ],
        'lora_bias': 'none',
        'lora_task_type': 'CAUSAL_LM',
        'use_dora': False,
        'use_rslora': True,

        'epochs': epochs,
```

```
        # Your current option-scoring trainer is effectively row-by-r
        # may not be used unless you patched it in. Keep this at 1.
        'gradient_accumulation_steps': 1,

        # DoRA usually benefits from a slightly lower LR than vanilla
        'lr': 5e-5,
        'weight_decay': 0.01,
        'max_grad_norm': 1.0,

        # These only matter if you added scheduler support.
        # If not, they will remain harmless config fields.
        'warmup_ratio': 0.05,
        'scheduler_type': 'cosine',

        **input_ablation_2,
    },
}

all_configs = build_configs_from_named_space(ablation_space, 'score')
display(pd.DataFrame(all_configs))
```

	training_objective	prompt_structure	context_mode	choice_format	ou
0	option_scoring	question_first	hint_only	letter_dot	

1 rows × 37 columns

```
fixed_context = {
    'MODEL_ID': MODEL_ID,
    'DEVICE': DEVICE,
    'torch_dtype': torch_dtype,
    'IMAGE_DIR': IMAGE_DIR,
    'train_df': train_df,
    'val_df': val_df,
    'test_df': test_df,
    'batch_size': batch_size,
    'clear_cuda_between_runs': clear_cuda_between_runs,
    'early_stopping': early_stopping,
    'epochs_per_val_check': epochs_per_val_check,
    'early_stopping_metric': early_stopping_metric,
    'early_stopping_val_examples': early_stopping_val_examples,
    # Save adapters once during val training so test submissions can
```

```
    'save_trained_adapters': bool(generate_submission),
    'ADAPTER_DIR': ADAPTER_DIR,
}

_param_rows = []
for cfg in all_configs:
    counts = count_trainable_params_for_config(cfg, {'MODEL_ID': MODEl
    _param_rows.append({
        'ablation_id': cfg['ablation_id'],
        'trainable_params': counts['trainable'],
        'total_params': counts['total'],
        'trainable_pct': round(float(counts['trainable_pct']), 4),
    })
display(pd.DataFrame(_param_rows).sort_values('ablation_id').reset_inc
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your setti
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to ac
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenti
config.json: 7.34k/? [00:00<00:00, 624kB/s]
`torch_dtype` is deprecated! Use `dtype` instead!
model.safetensors: 100% 1.02G/1.02G [00:04<00:00, 669MB/s]
Loading weights: 100% 489/489 [00:00<00:00, 1188.84it/s, Materializing param=model.vision_model.
generation_config.json: 100% 136/136 [00:00<00:00, 14.6kB/s]
```

	ablation_id	trainable_params	total_params	trainable_pct
0	score_dora	1040384	508522688	0.2046

```
results = []
val_preds = []
histories = []
submission_frames = []

config_by_id = {cfg['ablation_id']: cfg for cfg in all_configs}
all_ablation_ids = set(config_by_id.keys())

submission_requested_ids = [str(x) for x in (submission_ablation_ids
```

```
submission_selected_ids = set(submission_requested_ids) if submission_
submission_missing_ids = sorted(set(submission_requested_ids) - all_al
if submission_missing_ids:
    print('Skipping unknown submission ablation ids:', submission_miss

val_prediction_requested_ids = [str(x) for x in (val_prediction_ablat
val_prediction_selected_ids = set(val_prediction_requested_ids) if va
val_prediction_missing_ids = sorted(set(val_prediction_requested_ids)
if val_prediction_missing_ids:
    print('Skipping unknown validation-prediction ablation ids:', val_

for cfg in all_configs:
    ablation_id = cfg['ablation_id']
    summary, val_pred_df, history_df = run_lora_candidate_on_val(conf
    results.append(summary)
    val_preds.append(val_pred_df)
    if len(history_df):
        histories.append(history_df)

    # Generate submission immediately after training for selected abla
    if generate_submission and (ablation_id in submission_selected_ids
        adapter_path = ADAPTER_DIR / ablation_id
        test_pred_df, submission_df = run_test_submission_from_saved_a
            config=cfg,
            fixed_context=fixed_context,
            adapter_dir=adapter_path,
            save=save,
            out_dir=OUT_DIR,
            filename_prefix=f'mlp_attn_{ablation_id}_',
        )
        _sub = submission_df.copy()
        _sub['ablation_id'] = ablation_id
        submission_frames.append(_sub)
        print(f'Generated submission without retraining for: {ablation
        display(test_pred_df.head())
        display(submission_df.head())

    if clear_cuda_between_runs and torch.cuda.is_available():
        torch.cuda.empty_cache()

results_df = pd.DataFrame(results).sort_values('accuracy', ascending=F
val_predictions_df = pd.concat(val_preds, ignore_index=True) if val_pr
history_df = pd.concat(histories, ignore_index=True) if histories else
display(results_df)
```

```
if submission_frames:
    combined_submission_preview = pd.concat(submission_frames, ignore_
    display(combined_submission_preview.head())

if save and save_val_predictions:
    if val_prediction_requested_ids:
        selected_val_predictions_df = val_predictions_df[
            val_predictions_df['ablation_id'].isin(sorted(val_predict
        ].reset_index(drop=True)
    else:
        selected_val_predictions_df = val_predictions_df.copy()

    if len(selected_val_predictions_df):
        OUT_DIR.mkdir(parents=True, exist_ok=True)
        val_pred_path = OUT_DIR / 'mlp_attn_val_predictions.csv'
        selected_val_predictions_df.to_csv(val_pred_path, index=False
        print(
            'Saved validation predictions:',
            {
                'path': str(val_pred_path),
                'rows': int(len(selected_val_predictions_df)),
                'ablations': sorted(selected_val_predictions_df['abla
            },
        )
        display(selected_val_predictions_df.head())
    else:
        print('No validation predictions matched the selected ablatio
elif save_val_predictions and not save:
    print('save_val_predictions=True but save=False; skipping validat
```

processor_config.json: 100% 68.0/68.0 [00:00<00:00, 9.13kB/s]

chat_template.json: 100% 429/429 [00:00<00:00, 59.0kB/s]

preprocessor_config.json: 100% 486/486 [00:00<00:00, 59.4kB/s]

tokenizer_config.json: 28.2k/? [00:00<00:00, 3.28MB/s]

tokenizer.json: 3.55M/? [00:00<00:00, 115MB/s]

added_tokens.json: 4.74k/? [00:00<00:00, 571kB/s]

special_tokens_map.json: 1.07k/? [00:00<00:00, 129kB/s]

Loading weights: 100% 489/489 [00:00<00:00, 1167.83it/s, Materializing param=model.vision_mode

Score epoch 1/3: 100% 512/512 [26:31<00:00, 2.99s/it]

Score epoch 2/3: 100% 512/512 [20:35<00:00, 2.36s/it]

Score epoch 3/3: 100% 512/512 [20:35<00:00, 2.38s/it]

Val option scoring: 100% 256/256 [11:49<00:00, 2.57s/it]

Loading weights: 100% 489/489 [00:00<00:00, 1152.18it/s, Materializing param=model.vision_mode

Val option scoring: 100% 1008/1008 [47:28<00:00, 2.63s/it]

```
Generated submission without retraining for: score_dora
```

	id	pred_answer	num_choices
0	test_01918	3	4
1	test_02484	0	4
2	test_03843	2	4
3	test_02160	2	3
4	test_01805	0	2

	id	answer
0	test_01918	3
1	test_02484	0
2	test_03843	2
3	test_02160	2
4	test_01805	0

	training_objective	prompt_structure	context_mode	choice_format
0	option_scoring	question_first	hint_only	letter_dot

1 rows × 46 columns

	id	answer	ablation_id
0	test_01918	3	score_dora
2	test_03843	2	score_dora
3	test_02160	2	score_dora
4	test_01805	0	score_dora

Start coding or generate with AI.

```
Saved validation predictions: {'path': '/content/drive/MyDrive/DL_Fin
```

	id	pred_answer	num_choices	gold_answer	is_correct	ablatic
0	val_00570	2	3	0	0	score
1	val_03508	2	3	2	1	score
2	val_03587	1	3	0	0	score
3	val_00603	3	4	3	1	score